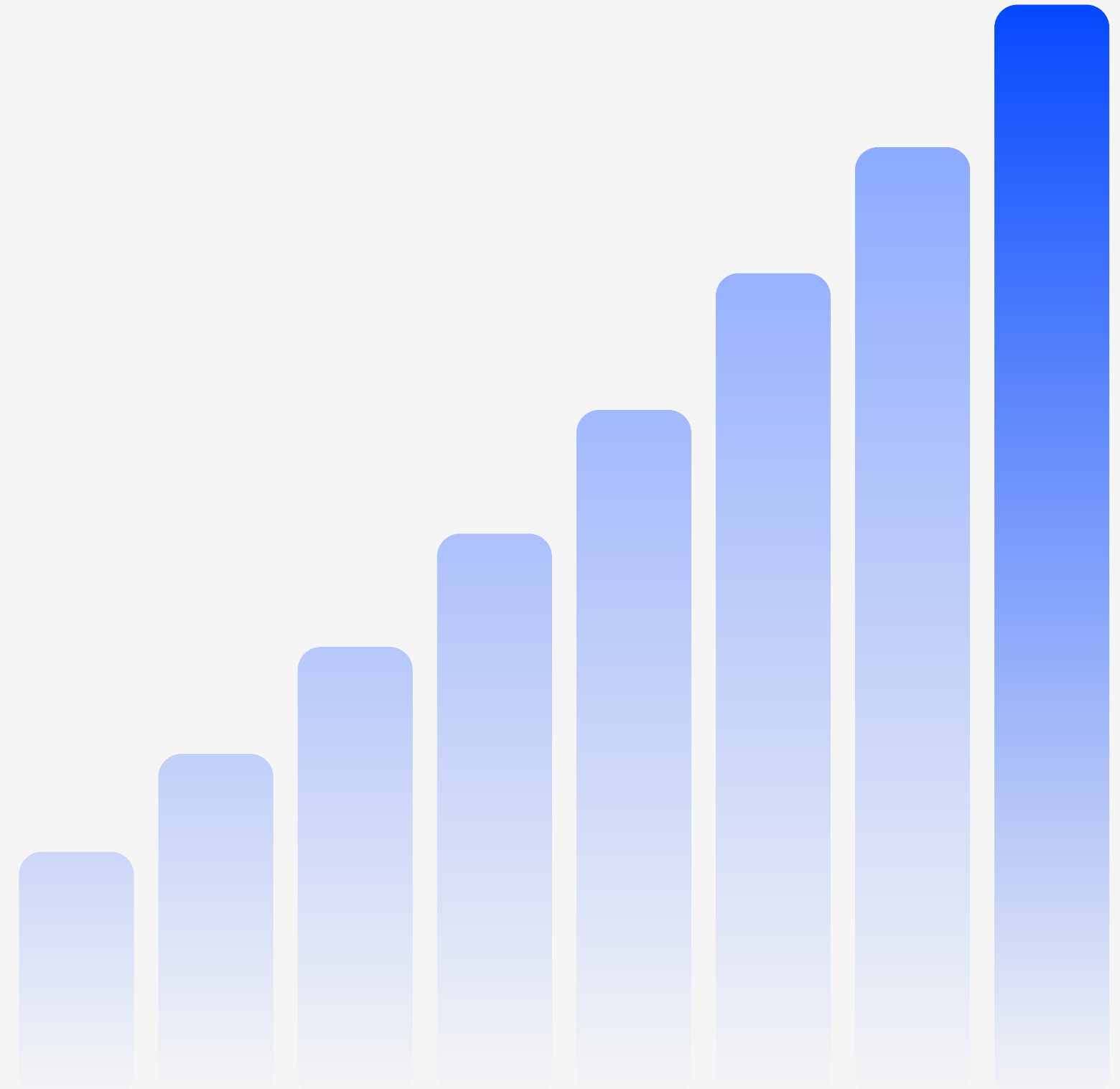


공공데이터 융합 풀스택 개발자 양성과정

통합구현



목 차

01

LinkScheduler 정승원

02

PostDTO 정승원

03

PostController 김정희

04

SecurityConfig 강현욱

05

LinkServiceImpl 박형규

06

LinkServiceTest 박형규

07

화면구현

01 | 코드구현 -LinkScheduler정승원

```
@Component  👤 kanghyunuk +1
@Slf4j
public class LinkScheduler {

    @Autowired
    private LinkService linkService;

    // TODO: ★학생 작업★ 주기적 연계 동기화
    // 1) 이 메서드 위에 @Scheduled(fixedDelayString = "${link.sync.delay:60000}")
    // 2) try { int n = linkService.syncPosts(); log.info("[Link] 배치 동기화 완료 - " + n + "건");
    //    catch (Exception e) { log.error("[Link] 배치 동기화 실패: " + e.getMessage()); }
    @Scheduled(fixedDelayString = "${link.sync.delay:60000}")  👤 jsh340866 +1
    public void scheduledSync() {
        // TODO: @Scheduled + syncPosts 호출 구현
        try {
            int n = linkService.syncPosts();
            log.info("[Link] 배치 동기화 완료 - " + n + "건");
        } catch (Exception e) {
            log.error("[Link] 배치 동기화 실패: " + e.getMessage());
        }
    }
}
```

1. @Scheduled가 붙은 scheduleSync() 메서드는 설정된 시간(기본 60초)마다 자동 실행됩니다.

2. 실행될 때 linkService.syncPosts()를 호출하여 게시글(링크) 데이터를 동기화하고, 처리 건수를 로그로 남깁니다.

3. 동기화 중 오류가 발생하면 try-catch에서 예외를 잡아 에러 로그를 기록하여 배치 작업이 중단되지 않도록 합니다.

```
public static PostDTO from(Post p) { 0개의 사용위치 👤js
//      throw new UnsupportedOperationException("TODO:
return PostDTO.builder()
    .id(p.getId())
    .userId(p.getUserId())
    .title(p.getTitle())
    .body(p.getBody())
    .createAt(p.getCreateAt())
    .build();
}
```

from(Post p) : Entity → DTO 변환

- Post엔티티 객체를 PostDTO 객체로 변환
- PostDTO.builder()를 이용하여 Entity의 각 필드를 DTO에 매핑
- DB 조회 결과를 화면 또는 API 응답용 DTO로 변환
- static 메서드로 구현되어 DTO 객체 생성 없이 호출 가능

```
public Post toEntity() { 0개의 사용위치 👤jsh340866 +1
//      throw new UnsupportedOperationException("TODO: ");
return Post.builder()
    .id(this.id)
    .userId(this.userId)
    .title(this.title)
    .body(this.body)
    .createdAt(LocalDate.now())
    .build();
}
```

toEntity() : DTO → Entity 변환

- Post.builder()를 이용하여 DTO의 각 필드를 Entity에 매핑
- id, userId, title, body 값을 Entity에 저장하고 createdAt은 현재 시간(LocalDate.now())으로 설정
- DB 저장(save()) 전에 DTO 데이터를 JPA Entity 형태로 변환할 때 사용
- 인스턴스 메서드로 구현되어 dto.toEntity() 형태로 호출 가능

03 | 코드구현 - PostController 김정희

```
@RestController
@Slf4j
@RequestMapping("/api/link")
@CrossOrigin(originPatterns = {"http://localhost:*"})
public class LinkController {

    @Autowired
    private LinkService linkService;

    @PostMapping(value = "/sync", produces = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<Map<String, Object>> sync() {
        int count = linkService.syncPosts();
        Map<String, Object> response = new HashMap<>();
        response.put("message", count + "연계 동기화 성공!");

        return ResponseEntity.status(HttpStatus.OK).body(response);
    }
}
```

컨트롤러 기본

@RestController로 기본 restful방식으로 적용

기본 엔드포인트 /api/link

localhost의 모든 포트는 권한 허용

LinkService 의존주입

수동 동기화(연계 수신)

엔드포인트 /api/link/sync, JSON 형태로 반환

linkService.syncPosts()로 저장 건수를 count에

받고 Map 타입으로 새로 생성 후 key에

"message"를 넣고 value에 count와 전달메시지를

넣고 ResponseEntity 타입으로 body에 넣어서

200 반환

03 | 코드구현 - PostController 김정희

```
@GetMapping(value = "/posts", produces = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<?> posts() {
    List<PostDTO> list = linkService.getPosts();
    return ResponseEntity.status(HttpStatus.OK).body(list);
}
```

```
@GetMapping(value = "/posts/{id}", produces = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<PostDTO> post(@PathVariable("id") Long id) {
    PostDTO postDTO = linkService.getPost(id);
    return ResponseEntity.status(HttpStatus.OK).body(postDTO);
}
```

연계 데이터 목록 재제공

엔드포인트 /api/link/posts, JSON 형태로 반환
linkService.getPosts()로 불러온 데이터를
List<PostDTO> 타입에 받고 ResponseEntitiy 타입으로 body에 넣어서 200 반환

단건 재제공

엔드포인트 /api/link/posts/{id}, JSON 형태로 반환
경로로 Long타입 id를 받아와
linkService.getPost(id)로 불러온 데이터를
PostDTO 타입에 받고 ResponseEntitiy 타입으로
body에 넣어서 200 반환

04 | 코드구현 - SecurityConfig 강현욱

```
@Configuration  📄 kanghyunuk
public class SecurityConfig {

    // TODO: ★학생 작업★ 연계 접근 보안 규칙 구현
    // - csrf 비활성화 (REST + httpBasic)
    // - 인가: POST /api/link/sync → hasRole("ADMIN")
    //       /api/link/**      → authenticated()
    //       그 외             → permitAll()
    // - 인증: httpBasic 사용
    @Bean  📄 kanghyunuk
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf( CsrfConfigurer<HttpSecurity> csrf -> csrf.disable())
            .authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
                .requestMatchers(org.springframework.http.HttpMethod.POST, _patterns: "/api/link/sync").hasRole("ADMIN")
                .requestMatchers( _patterns: "/api/link/**").authenticated()
                .anyRequest().permitAll()
            )
            .httpBasic(org.springframework.security.config.Customizer.withDefaults());
        return http.build();
    }
}
```

SECURITY

- 클라이언트가 API 요청을 보낸다.
- Spring Security가 요청 URL을 확인한다.
- POST /api/link/sync 요청이면 ADMIN 권한을 검사한다.
- /api/link/** 요청이면 로그인 여부를 검사한다.
- 그 외 요청은 누구나 접근 가능하다.
- 인증은 HTTP Basic 방식으로 수행된다.
- 조건을 만족하면 Controller로 요청이 전달된다.

04 | 코드구현 - SecurityConfig 강현욱

```
// TODO: 인메모리 사용자 - user(USER) / admin(ADMIN), 비밀번호 "1234"
@Bean
public InMemoryUserDetailsManager userDetailsService() {
    UserDetails user = User.withUsername("user")
        .password(passwordEncoder().encode("1234")).roles("USER").build();
    UserDetails admin = User.withUsername("admin")
        .password(passwordEncoder().encode("1234")).roles("ADMIN").build();
    return new InMemoryUserDetailsManager(user, admin);
}
```

SECURITY

- 서버 시작 시 USER와 ADMIN 계정을 생성한다.
- 계정 정보를 메모리에 저장한다.
- 사용자가 로그인 요청을 보낸다.
- Spring Security가 InMemoryUserDetailsManager에서 사용자 정보를 조회한다.
- 비밀번호를 검증한다.
- 인증 성공 시 권한(USER 또는 ADMIN)을 부여한다.
- 설정된 URL 접근 권한을 검사한다.

05 | 코드구현 - LinkServiceImpl 박형규

```
// TODO: 외부 API 연계(수신) → 변환 → DB 저장
// - @Transactional 적용
// - RestTemplate + UriComponentsBuilder 로 (apiBase + "/posts", ?_limit=10) GET 호출
//   예: restTemplate.exchange(url, HttpMethod.GET, null, PostDTO[].class)
// - 상태코드가 HttpStatus.OK 가 아니거나 body 가 null 이면 MyBizException 발생 (연계 장애)
// - 응답 PostDTO[] 를 들며 createdAt=now() 설정 후 postRepository.save(dto.toEntity())
// - 저장 건수(int) 반환
@Override 1개 사용 위치 8 parkhyeonggyu15 +1
@Transactional
public int syncPosts() {
    String url = UriComponentsBuilder.fromUriString(apiBase + "/posts")
        .queryParams(name: "_limit", values: 10)
        .toUriString();

    RestTemplate restTemplate = new RestTemplate();
    ResponseEntity<PostDTO[]> response = restTemplate.exchange(url, HttpMethod.GET, requestEntity: null, PostDTO[].class);

    if (response.getStatusCode() != HttpStatus.OK || response.getBody() == null) {
        throw new MyBizException("외부 API 연계 실패: " + url);
    }

    PostDTO[] posts = response.getBody();
    for (PostDTO dto : posts) {
        dto.setCreatedAt(LocalDate.now());
        postRepository.save(dto.toEntity());
    }

    return posts.length;
}
```

apiBase에 /posts를 붙이고 _limit=10 쿼리를 추가해 URL을 만든 뒤, RestTemplate으로 GET 요청을 보내 PostDTO[]로 응답을 받는다.
응답 상태가 OK가 아니거나 바디가 null이면 예외를 던지고, 정상이면 각 DTO에 현재 시각을 세팅해 DB에 저장한 후 저장된 게시물 수를 반환한다.

```
// TODO: DB 저장된 연계 데이터 목록 재제공
// - @Transactional(readOnly = true)
// - findAll() → PostDTO.from 으로 매핑하여 List 반환
@Override 1개 사용 위치 2 parkhyeonggyu15 +1
@Transactional(readOnly = true)
public List<PostDTO> getPosts() {
    return postRepository.findAll().stream() Stream<Post>
        .map(PostDTO::from) Stream<PostDTO>
        .collect(Collectors.toList());
}
```

DB에서 모든 Post를 조회한 뒤 스트림으로 변환하고, 각 Post를 PostDTO::from으로 매핑한 후 리스트로 수집해 반환한다.

05 | 코드구현 - LinkServiceImpl 박형규

```
// TODO: 단건 재제공
// - @Transactional(readOnly = true)
// - findById(id) (없으면 MyBizException) → PostDTO.from
@Override @개인의 사용위치  parkhyeonggyu15 +1
@Transactional(readOnly = true)
public PostDTO getPost(Long id) {
    return postRepository.findById(id).Optional<Post>
        .map(PostDTO::from).Optional<PostDTO>
        .orElseThrow(() -> new MyBizException("게시글을 찾을 수 없습니다. id=" + id));
}
```

id로 Post를 조회하고, 존재하면 PostDTO::from으로 매핑해 반환하고 없으면 MyBizException을 던진다.

```
@SpringBootTest @kanghyunuk +1
class LinkServiceTest {

    @Autowired
    private LinkService linkService;

    // TODO: 연계 동기화 정상 테스트 (NCS 3 - 연계 테스트)
    // - linkService.syncPosts() 호출 (외부망 필요)
    // - 반환된 저장 건수가 0 보다 큰지 assertTrue
    // - 이어서 linkService.getPosts() 가 비어있지 않은지 확인
    @Test @parkhyeonggyu15 +1
    void syncPosts_정상() {
        int savedCount = linkService.syncPosts();
        assertTrue(condition: savedCount > 0);

        assertFalse(linkService.getPosts().isEmpty());
    }
}
```

syncPosts()를 호출해 저장된 건수가 0보다 큰지 확인
하고, getPosts()로 조회한 목록이 비어있지 않은지 검
증한다.

07 | 구현화면

POST

http://localhost:8090/api/link/sync

Send

Params

Authorization

Headers (11)

Body

Pre-request Script

Tests

Settings

Cookies

Type

Basic Au...

The authorization header will be automatically generated when you send the request.
[Learn more about authorization](#)

Username

admin

Password

1234

Store your secrets with end-to-end encryption locally using Vault.

Store in Vault

Body

Cookies

Headers (14)

Test Results

200 OK 278 ms 463 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

1

2

3

"message": "10연계 동기화 성공!"

수동 동기화

POST

http://localhost:8090/api/link/sync

07 | 구현화면

GET

http://localhost:8090/api/link/posts

Send

Params

Authorization

Headers (11)

Body

Pre-request Script

Tests

Settings

Cookies

Query Params

	Key	Value	Bulk Edit
--	-----	-------	-----------

Body

Cookies

Headers (14)

Test Results

200 OK

149 ms

3.1 KB

Save Response

Pretty

Raw

Preview

Visualize

JSON

```
1 [
2   {
3     "id": 1,
4     "userId": 1,
5     "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6     "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae
7       ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto",
8     "createAt": "2026-06-24T15:50:52"
9   },
10  {
11    "id": 2,
12    "userId": 1,
13    "title": "qui est esse",
14    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat
15      blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui
16      neque nisi nulla",
17    "createAt": "2026-06-24T15:50:52"
18  },
19  {
20    "id": 3,
21    "userId": 1,
22    "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
23    "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel
24      accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut",
25    "createAt": "2026-06-24T15:50:52"
26  },
27  {
28    "id": 4,
29    "userId": 1,
30    "title": "eum et est occaecati",
31    "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum
```

전체목록 조회

GET

http://localhost:8090/api/link/posts

07 | 구현화면

The screenshot shows a REST client interface with the following details:

- Request:** Method: GET, URL: http://localhost:8090/api/link/posts/1
- Response:** Status: 200 OK, Time: 77 ms, Size: 731 B
- Response Body (JSON):**

```
1 {
2   "id": 1,
3   "userId": 1,
4   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut
        quas totam\nnostrum rerum est autem sunt rem eveniet architecto",
6   "createAt": "2026-06-24T15:51:52"
7 }
```

단건 조회

GET

http://localhost:8090/api/link/posts/1

07 | 구현화면

배치 자동 동기화

```
where
  id=?
2026-06-24T15:51:52.007+09:00 INFO 5360 --- [demo] [   scheduling-1] c.example.demo.Scheduled.LinkScheduler : [Link] 배치 동기화 완료 - 10건
Hibernate:
select
  p1 0.id.
where
  id=?
2026-06-24T15:52:52.199+09:00 INFO 5360 --- [demo] [   scheduling-1] c.example.demo.Scheduled.LinkScheduler : [Link] 배치 동기화 완료 - 10건
Hibernate:
select
  title=?,
  user_id=?
where
  id=?
2026-06-24T15:53:52.418+09:00 INFO 5360 --- [demo] [   scheduling-1] c.example.demo.Scheduled.LinkScheduler : [Link] 배치 동기화 완료 - 10건
```